

Schema Validation in MongoDB

MongoDB is known for its flexibility - you can store documents without predefined structures. However, as your application grows, you might want to ensure your data follows certain rules. This is where schema validation comes in.

How Schema Validation Works

When you create a collection with validation, MongoDB will check every new document (and updates to existing ones) against your rules. Here's what the basic structure looks like:

```
db.createCollection("collectionName", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["field1", "field2"],
      properties: {
        field1: { type: "string" },
        field2: { type: "number" }
      }
    }
  }
})
```

JavaScript

The `$jsonSchema` keyword tells MongoDB that we're using JSON Schema validation. Inside this schema, we define our rules using various building blocks.

Building Blocks of Validation

The most fundamental components are:

First, we specify which fields are mandatory using `required`. These fields must be present in every document.

Next, we define `properties` - this is where we describe what each field should look like. For each property, we can specify its type and additional rules. For example, if you're storing someone's age, you might want to ensure it's always a number and perhaps even set a minimum / maximum value.

Let's look at how we handle more complex structures:

Nested Objects

Sometimes your data has natural hierarchies. For instance, an address isn't just one piece of information - it has streets, cities, and zip codes. Here's how you validate nested structures:

```
properties: {
  address: {
    bsonType: "object",
    required: ["city"], // City is mandatory in addresses
    properties: {
      city: { type: "string" },
      zip: { type: "string" }
    }
  }
}
```

JavaScript

Working with Arrays

Arrays are perfect for storing lists of things - like a person's phone numbers or a product's tags. MongoDB lets you validate both the array itself and its contents:

```
properties: {
  phoneNumbers: {
    bsonType: "array",
    minItems: 1, // Must have at least one phone number
    items: {
      bsonType: "string", // Each phone number must be a string
      pattern: "^\\d{10}$" // Must be exactly 10 digits
    }
  }
}
```

JavaScript

Fine-Tuning Validation Behavior

MongoDB gives you control over how strict your validation should be. You can set two important behaviors:

The `validationAction` determines what happens when a document fails validation: - "error" (default): Reject the document completely - "warn": Accept the document but log a warning (great during development!)

The `validationLevel` controls when validation happens: - "strict" (default): Check all inserts and updates - "moderate": Skip validation for existing documents that don't match the schema

Remember that validation only happens when documents are modified or inserted. Existing documents won't be validated until you try to update them. This makes it safe to add validation to collections that already contain data.

Through schema validation, MongoDB offers a balance between flexibility and control. You can start with a loose schema during early development, then gradually add more validation rules as your application's needs become clearer. This progressive approach to data quality helps ensure your database remains both reliable and adaptable.